

backpropagation_practice.py

```
1 import math
2 import numpy as np
3
4 def matrix_mult(matrix_a, matrix_b): #TODO: use strassen algorithm instead to make
    faster
5     is_array = False
6     a = len(matrix_a)
7
8     if type(matrix_a[0]) != list:
9         is_array = True
10        a = 1
11        matrix_a = [matrix_a]
12
13    if type(matrix_b[0]) != list:
14        matrix_b = [matrix_b]
15
16    matrix_r = [[0 for i in range(len(matrix_b[0]))] for j in range(a)]
17
18    for i in range(len(matrix_a)):
19        for j in range(len(matrix_b[0])):
20            for k in range(len(matrix_a[0])):
21                matrix_r[i][j] += matrix_a[i][k] * matrix_b[k][j]
22
23    if is_array:
24        matrix_r = matrix_r[0]
25
26    return matrix_r
27
28 def matrix_add(matrix_a, matrix_b):
29     new_matrix = []
30     for e in range(len(matrix_a)):
31         new_matrix.append(matrix_a[e] + matrix_b[e])
32     return new_matrix
33
34 class Layer():
35     def __init__(self, num_neurons, prev_layer, layer_weights, biases):
36         self.num_neurons = num_neurons
37         self.prev_layer = prev_layer
38         self.layer_weights = layer_weights
39         self.biases = biases
40         self.outputs = []
41         self.drop_store = {
42             'cur': {},
43             'for': {}
44         }
45
46     def setLayerOuts(self):
```

```
47     net = matrix_add(matrix_mult(self.prev_layer.outputs, self.layer_weights),
self.biases)
48     out = []
49     for n in net:
50         o = 1 / (1 + pow(math.e, -n))
51         out.append(o)
52     self.outputs = out
53
54     def insert(self, n, forward=False):
55         if forward:
56             # insert values into the forward layer
57             self.layer_weights.insert(n, self.drop_store['for'][n]['w'])
58
59             # call insert for the back layer
60             self.prev_layer.insert(n)
61         else:
62             self.num_neurons += 1
63
64             # grab dropped values from storage
65             values = self.drop_store['cur'].pop(n)
66
67             # put values back into matrices
68             self.outputs.insert(n, values['out'])
69             self.biases.insert(n, values['b'])
70
71             for r, row in enumerate(self.layer_weights):
72                 row.insert(n, values['w'][r])
73
74     def pop(self, n, forward=False):
75         if forward:
76
77             # pop weights for the forward layer
78             self.drop_store['for'].update({
79                 n : {
80                     'w': self.layer_weights.pop(n)
81                 }
82             })
83
84             # pop values for the back layer
85             self.prev_layer.pop(n)
86         else:
87             self.num_neurons -= 1
88
89             # remove pop values and store as new variables
90             out = self.outputs.pop(n)
91             b = self.biases.pop(n)
92
93             pop_weights = []
94             for row in self.layer_weights:
95                 w = row.pop(n)
```

```

96         pop_weights.append(w)
97
98     # store dict of values
99     self.drop_store['cur'].update({
100         n: {
101             'w': pop_weights,
102             'b': b,
103             'out': out
104         }
105     })
106
107 class Network():
108     def __init__(self, sys_inputs, target_output, learning_rate, error_threshold):
109         self.sys_inputs = sys_inputs
110         self.target_output = target_output
111         self.error_threshold = error_threshold
112         self.network_weights = []
113         self.network_biases = []
114         self.layers = []
115         self.learning_rate = learning_rate
116
117     def truncate(self, number, decimals):
118         factor = pow(10, decimals)
119         return int(number * factor) / factor
120
121     def size(self, matrix):
122         return len(matrix) * len(matrix[0])
123
124     def getError(self):
125         sum = 0
126         for i in range(len(self.target_output)):
127             sum += (1 / len(self.target_output)) * pow((self.target_output[i] -
self.layers[-1].outputs[i]), 2)
128         return sum
129
130     def addLayer(self, num_neurons, layer_type='hidden'):
131         if layer_type == 'input':
132             prev_layer_neurons = len(self.sys_inputs)
133         else:
134             prev_layer_neurons = self.layers[-1].num_neurons
135
136         # create random weight values
137         new_weights = []
138         for r in range(prev_layer_neurons):
139             row = []
140             for w in range(num_neurons):
141                 row.append(self.truncate((np.random.rand() + 0.01) *
(np.random.randint(5) + 0.01), 3)) #TODO: reimplement later, also don't use np
142             new_weights.append(row)
143         self.network_weights.append(new_weights)
144

```

```

145         # create random bias values
146         new_biases = []
147         for b in range(num_neurons):
148             new_biases.append(self.truncate(np.random.rand() * np.random.randint(10),
3)) #TODO: reimplement later, also don't use np
149         self.network_biases.append(new_biases)
150
151         # make a new layer with the weights and biases
152         if layer_type == 'input':
153             ghost_layer = Layer(len(self.sys_inputs), None, None, None)
154             ghost_layer.outputs = self.sys_inputs
155             self.layers.append(Layer(num_neurons, ghost_layer, new_weights,
new_biases))
156         else:
157             self.layers.append(Layer(num_neurons, self.layers[-1], new_weights,
new_biases))
158
159         self.layers[-1].setLayerOuts()
160
161     def cumulative_partial(self, layer_num, w=None, b=None):
162         layer = self.layers[layer_num]
163
164         if w is not None:
165             # dNet_dWeight
166             dInit = layer.prev_layer.outputs[int((w % self.size(layer.layer_weights))
/ layer.num_neurons)]
167
168             # dOut_dWeight
169             z = w % layer.num_neurons
170             out = layer.outputs[z]
171             dInit *= out * (1 - out)
172         else:
173             # dOut_dBias
174             dInit = layer.biases[b]
175             dInit *= layer.outputs[b] * (1 - layer.outputs[b])
176
177         path = []
178
179         if layer != self.layers[-1]:
180             n = int((w / self.layers[layer_num].prev_layer.num_neurons) if (w is not
None) else b)
181             path.append(self.layers[layer_num + 1].layer_weights[n])
182
183         for l in range(layer_num + 1, len(self.layers)):
184             current_layer = self.layers[l]
185             m = []
186
187             # append a diagonal matrix for dOut_dNet
188             for o, out in enumerate(current_layer.outputs):
189                 sub_m = []
190                 for j in range(current_layer.num_neurons):

```

```

191         if j == 0:
192             sub_m.append(out * (1 - out))
193         else:
194             sub_m.append(0)
195         m.append(sub_m)
196     path.append(m)
197
198     # append the weight matrix for the layer
199     if current_layer != self.layers[-1]:
200         path.append(self.layers[l+1].layer_weights)
201     else:
202         break
203
204     #dE_dOut
205     if layer != self.layers[-1]:
206         m = []
207         for o, out in enumerate(self.layers[-1].outputs):
208             m.append([(-1 * (self.target_output[o] - out))])
209         path.append(m)
210     else:
211         # if parameter in last layer error dependent on one neuron
212         idx = (w if w is not None else b) % self.layers[-1].num_neurons
213         resid = -1 * (self.target_output[idx] - self.layers[-1].outputs[idx])
214         path.append(self.layers[-1].outputs[idx] * (1 -
self.layers[-1].outputs[idx]))
215         path.append(resid)
216
217     # multiply the matrix elements of path together to get a scalar
218     if layer != self.layers[-1]:
219         starter = []
220         for e in path[0]:
221             starter.append(e * dInit)
222
223         running = starter
224         for element in range(len(path) - 2):
225             running = matrix_mult(running, path[element + 1])
226
227         total = 0
228         for r, run in enumerate(running):
229             total += (run * path[-1][r][0])
230     else:
231         total = dInit * path[0] * path[-1]
232
233     return total
234
235     def updateAll(self):
236         # clear the new weight and bias network arrays
237         new_network_weights = []
238         new_network_biases = []
239

```

```

240     for l, layer in enumerate(self.layers):
241
242         # calculate new weight by  $w' = w - a * dE_{dw}$ 
243         new_layer_weights = []
244         for rw, row in enumerate(layer.layer_weights):
245             new_row = []
246             for w, weight in enumerate(row):
247                 new_row.append(weight - self.learning_rate *
self.cumulative_partial(l, w=(rw * layer.num_neurons) + w))
248             new_layer_weights.append(new_row)
249
250         # calculate new bias by  $b' = b - a * dE_{db}$ 
251         new_layer_biases = []
252         for b, bias in enumerate(layer.biases):
253             new_layer_biases.append(bias - self.learning_rate *
self.cumulative_partial(l, b=b))
254
255         new_network_weights.append(new_layer_weights)
256         new_network_biases.append(new_layer_biases)
257
258     self.network_weights = new_network_weights
259     self.network_biases = new_network_biases
260
261     # distribute weights to appropriate layer
262     for l, layer in enumerate(self.layers):
263         layer.layer_weights = self.network_weights[l]
264         layer.biases = self.network_biases[l]
265         layer.setLayerOuts()
266
267     def minimizeError(self):
268         current_error = self.getError()
269
270         # update all parameters until the error is below the threshold
271         while current_error >= self.error_threshold:
272             self.updateAll()
273             current_error = self.getError()
274
275     def drop(self, l, n):
276         #TODO: make more advanced like deciding what neurons are dropped
277         # make sure to set layer outs here
278
279         # Permanent cond
280         # - can never remove all neurons
281
282         # Begin
283         # - generate number of neurons remove based on randomness and size factors
284         # - ea. neuron equal probability remove
285         # - Number the neurons in the network (not per layer)
286         # - Randomly select
287         # - Neurons in layer and pos. Surrounding one that was removed have less
likely chance to get chosen remove.

```

```
288
289
290     self.updateAll()
291
292     self.layers[l+1].pop(n, True)
293     self.updateAll()
294
295     self.layers[l+1].insert(n, True)
296     self.updateAll()
297
298 def main():
299     # define constants for the network
300     learning_rate = 1
301     sys_input = [0.05, 0.10]
302     target_output = [0.01, 0.99]
303     error_threshold = 0.00001
304
305     # create the architecture for a layered network
306     network = Network(sys_input, target_output, learning_rate, error_threshold)
307     network.addLayer(2, layer_type='input')
308     network.addLayer(2)
309     network.addLayer(len(target_output))
310
311     # testing if can properly update when drop neurons
312     network.drop(1, 0)
313
314
315     # optimize the parameters of the network for the target
316     print(f'Original Error: {network.getError()}')
317     network.minimizeError()
318     print(f'Updated Error: {network.getError()}')
319
320     print('finished')
321
322 if __name__ == "__main__":
323     main()
324
```